

AlphaSight

实证洞察与证伪挑战

基于美股多源金融语料的 研报生成 + 错误检测 比赛

任务 · 产物

围绕 50 支美股大盘股的多源语料，做一个既能 **写研报** 也能 **审研报** 的 Agent。

- **Generate**: 给定 topic，写一份扎根语料、有立场的研究报告（Markdown）
- **Review**: 给定一份已有研报，找出其中的事实/逻辑错误

你要交的产物：

output/generate/*.md	10 份研报（6 固定 + 4 自由）
output/review.jsonl	val 集 20 题的审稿结果 ← leaderboard 看这个
submission/	整套代码 + 答辩材料

详细约定见 `Summer-Camp-Projects/README.md`，这里只讲会用到的关键点。

开工 3 步

```
# 1. qz 平台选 alphasight:v1 镜像启实例；数据已在共享路径
cp -r /inspire/qb-ilm2/project/26summer-camp-03/public/dataset \
    Summer-Camp-Projects/

# 2. 起 vLLM (任意 Qwen 模型都行, 参数细节见 deploy/README.md)
vllm serve <model-path> --tensor-parallel-size 4 --port 8000 &

# 3. 跑通 baseline (-example 走 ExampleSubmission, 不必先写 Submission)
python reference_submission/run.py review --example \
    --requests reference_submission/problem/review_sample.jsonl
```

环境变量按 `.env.example` 填 (数据路径 + LLM 端点)。每队可使用 4× H200。

最终评测无外网：模型权重、第三方依赖都要打进 `submission/`。

你要实现的： `Submission` 类的两个方法

```
from schemas import GenerateRequest, ReviewRequest, Report, ReviewIssue

class Submission:
    def generate(self, request: GenerateRequest) -> Report: ...
    def review(self, request: ReviewRequest) -> list[ReviewIssue]: ...
    # ReviewIssue = {quote: str, reason: str}
    # 找不到错误就返回 []
```

参考实现 `ExampleSubmission` (`catalog` → `BM25` → 单轮 LLM) 只为跑通流水线。最快起步：

```
class Submission(ExampleSubmission): # 拿基线行为
    pass                             # 再逐方法 override
```

工具层 `llm.chat()` / `catalog.py` / `submission.yaml` 一应俱全——直接看 `reference_submission/` 里的代码。

数据 · 题库

6 维度语料 (~2.5 GB, 2025—2026)

filings/	SEC 10-K / 10-Q / 8-K ...
news/	Polygon + Finnhub 财经新闻
research/	结构化分析师数据
social/	Twitter cashtag 日聚合
prices/	日 OHLCV
prices_minute/	分钟 OHLCV
catalog.jsonl	corpus 元数据索引

字段细节: `dataset/README.md`。

Review 题分三集

集合	题量	GT	用途
train	20	✓	本地调试
val	20	×	提交评测、上 leaderboard
test	20	×	比赛末切换

Generate 10 道题

- 6 固定题 (NVDA/PFE/META/GOOGL/NKE/银行业归因)
- 4 自由题 (自填 topic + 立场 + 多源支撑)

Review 评分 — F2 + issue-level 语义匹配

判分流程：判分 LLM 逐题在你的 issues 和 GT issues 之间做 1-to-1 配对。

- `quote` 之间是**语义匹配**，文字不必一致，能让模型看出你指报告里哪句话即可
- 配上 = TP；漏掉的 GT = FN；多报的 = FP；control 题（GT 为空）答空 = TN

主分 F2 (Recall 权重 = 4×Precision)

$$F_2 = \frac{5PR}{4P + R}$$

漏掉真错的代价 >> 多报的代价。

Worked example (Claude baseline)

```
TP=21  FN=14  FP=5  (35 GT)
P = 21/26 = 0.808
R = 21/35 = 0.600
F2 = 5·P·R / (4P+R) = 0.633
总分 = 63.3
```

Leaderboard 实时反馈，多次提交按最新分覆盖。判分 LLM 用 GLM-5.1。

注意：Leaderboard 上的分数仅供参考。最终 Review 成绩以你提交代码后，助教组运行你的 `submission/` 在 test 集上的分数为准——所以代码必须能离线跑通。

总成绩 = 三部分

Review 自动评测

20%

leaderboard F2 分

Generate 评审

20%

赛后助教用前沿 Agent 做
LLM-as-Judge

材料 + 答辩

60%

评审组现场综合打分

上交 & 注意事项

提交：通过学院统一平台上传 `review.jsonl` → 平台异步评测 ~60—120 秒出分 → 入 leaderboard。可多次提交。

容易踩的坑

- `request_id` 必须等于报告 stem (如 `report_01`) ——对不上 = 漏检
- `quote` 应从原文摘 (判分时仍按语义匹配 GT, 但要让模型看出你指哪处)
- 建议避免将任何知识、参考数据硬编码到代码中
- **不要伪造数字**——review 通道会把假数据全捞出来
- `submission/` 必须能**离线跑** (权重、依赖都打包, 结构见 `README.md §7`)

答疑：现场助教直接找 · 微信群 · qz 使用文档另行下发

开始动手吧

Good hunting.